



Getting Started with Agentic AI and Google ADK

A practical introduction to building intelligent, action-taking AI agents using Google's Agent Development Kit — from core concepts to robust production patterns.

DEVELOPER GUIDE

GOOGLE ADK

Before We Start — A Quick Sense Check

Take a moment with these questions. They're anchors, not gates. The more familiar these ideas feel, the faster the concepts ahead will click.

1

LLM Basics

- What is a large language model?
- Why use an LLM instead of a fixed rule-based system?
- What does it mean when an LLM predicts the next token?
- Why can the same prompt produce different answers at different times?

2

Prompting

- What is a prompt?
- Why do we use prompts to guide model behavior?
- What is a system prompt?
- Why does prompt wording affect the answer we get?

3

GenAI Workflows

- What is generative AI?
- Why use GenAI for open-ended tasks instead of hard-coded flows?
- Difference between a chat assistant and a task-completing assistant?
- Why do some GenAI systems need tools, memory, or routing?

4

APIs and Data

- What is an API?
- Why do GenAI systems often call APIs or external services?
- What is JSON?
- Why is structured data useful when a model needs to act on information?

1. What Is an AI Agent?

The Core Question

How is an AI agent different from just calling a language model?

A language model takes a prompt and returns text — one shot, no memory, no actions. An **AI agent** wraps that same model inside a loop:

Perceive → Reason → Act → Observe → Reason again → Respond

The loop keeps running until the agent decides the task is complete.

What Makes Something an "Agent"

→ Calls Tools

Functions that read data, write data, or trigger external actions.

→ Maintains State

Remembers what happened earlier in the conversation.

→ Makes Decisions

Chooses which tool to use, when, and with what inputs.

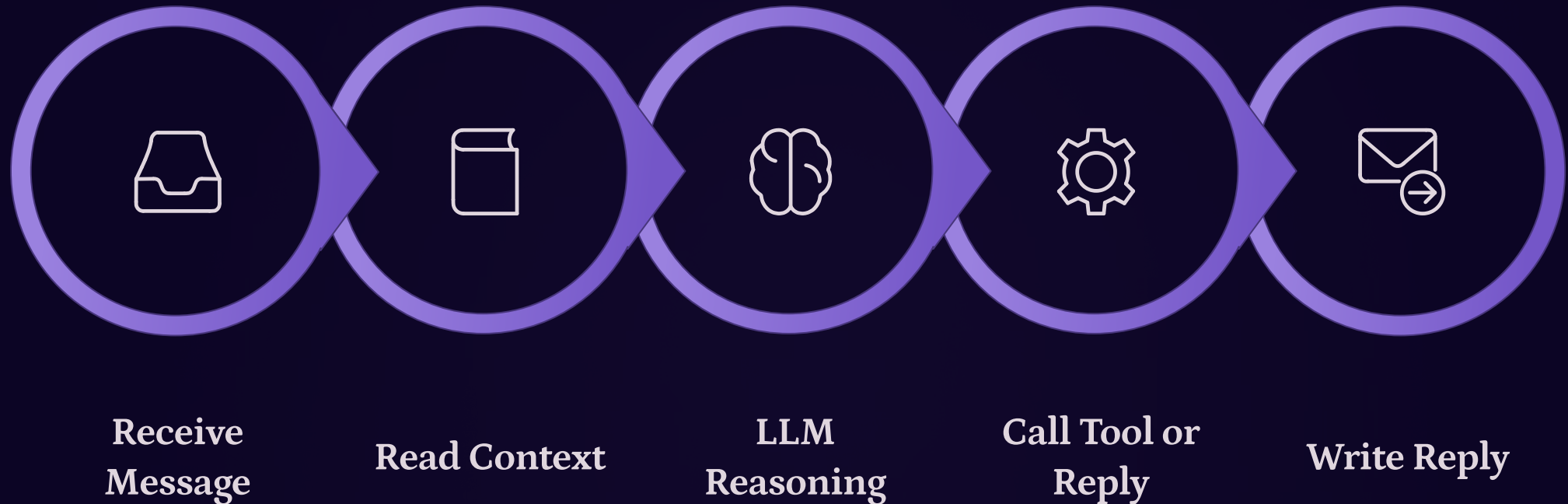
→ Can Delegate

Hands sub-tasks to other specialized agents.

 The model is the brain. The agent framework is the body — perception, memory, hands, and voice.

2. What Is the Agent Loop?

What actually happens inside an agent between receiving a message and sending a reply?



Why This Matters

- Steps 3–4 can repeat **multiple times** in a single turn — this is multi-step reasoning.
- The agent owns the loop; the framework enforces it.
- Every step is observable: which tool was called, with what inputs, and what came back.

❑ A plain LLM skips steps 2, 3, and 4 entirely. **The loop is what separates "chat" from "action."**

3. What Is Google ADK?

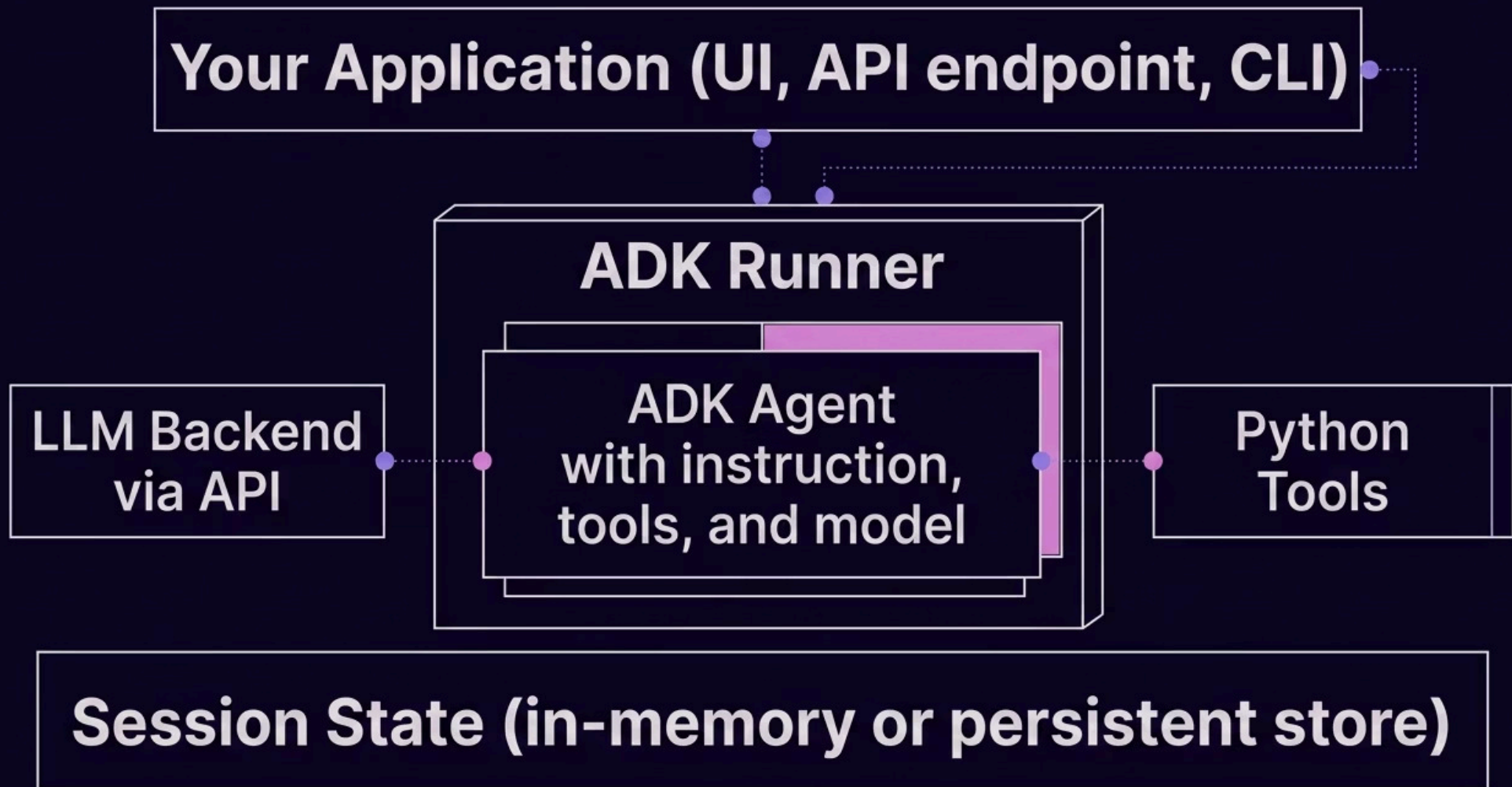
Google ADK is an open-source Python framework that provides the agent loop out of the box. Here's what each component does — and what ADK deliberately leaves to you.

Component	What It Does
Agent class	Defines an agent — model, instruction, tools, and optional sub-agents
@tool decorator	Turns any Python function into a tool the LLM can choose to call
SessionService	Manages conversation history and key-value state across turns
Runner	Executes the agent loop — receives messages, drives reasoning, returns responses
ADK Web	Browser UI for interactive testing and step-by-step trace inspection

 **What ADK deliberately does not do:** It does not pick your model, define your tools, or own your data. You choose the LLM (Gemini, OpenAI, Anthropic, and others), write the tools as plain Python functions, and configure your own session storage. ADK is the runtime — you provide the intelligence.

4. How Does ADK Fit Into the Broader Stack?

Where does ADK sit relative to LLMs, tools, and the application layer?



- ❏ **The LLM reasons.** Your tools act. Your state persists. ADK connects all three and runs the loop between them.

Key Architectural Principle

Your application never talks directly to the LLM — it talks to the Runner. The Runner manages all orchestration, tool dispatch, and state persistence. This separation keeps your application code clean and your agent logic portable.

5. What Is an Agent Instruction?

The instruction is the agent's **operating contract** — it defines who the agent is, what it handles, and how it should behave in every situation.

1. Persona

Who the agent is.

"You are a support assistant for an online electronics store."

2. Domain Scope

What is in and out of bounds.

"You help with orders, products, returns, and technical guidance only."

3. Behavioral Rules

How to handle specific situations.

"If you do not have data to answer, say so. Never fabricate details."

4. Tone and Style

How to communicate.

"Keep responses concise, clear, and professional."

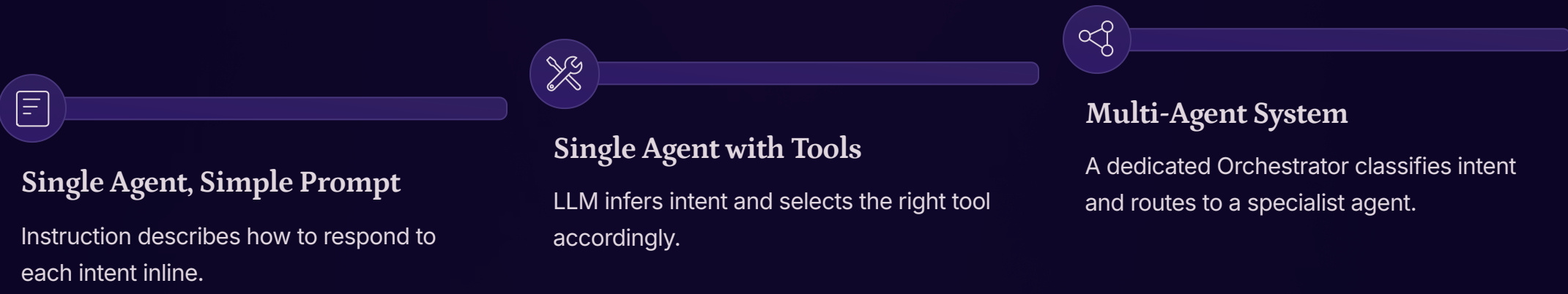
⚠ **What breaks without a good instruction:** The agent answers anything — not just in-scope queries. It may hallucinate data it was never given. Tone and persona become inconsistent across turns. The instruction is not just a prompt — it is the agent's constitution.

6. What Is Intent Modeling?

An **intent** is the goal beneath a user's words — the "why" behind the message. Users rarely express intent in structured terms. Here's how the same message maps to a concrete need:

What the User Says	What They Actually Want
"I haven't received my stuff"	Order status / tracking
"Can I get my money back?"	Return or refund
"What's a good option under 20k?"	Product recommendation
"It stopped working after a week"	Technical support or complaint
"Hi, can someone help me?"	General greeting and triage

How Intent Handling Evolves With Complexity



7. What Is ADK Web?

ADK Web is a local browser interface built into the ADK toolkit. It is designed for **development and debugging** — not for production use.

What ADK Web Shows You

- The agent's full reply for each turn
- Session state — what the agent currently holds in memory
- Tool calls made — the input sent and the output received
- Model reasoning steps where visible
- Structured logs per message, per tool, per state change

When to Use It — and When Not To

✓ **Right tool:** Validating a new agent definition, iterating on instructions, debugging unexpected tool calls, or demoing to stakeholders.

⚠ **Not enough for:** Performance testing or end-user delivery — ADK Web is not styled, secured, or hardened for production.

ADK Web is your agent's flight simulator. **Validate everything here before deploying it anywhere else.**

8. How Does Session State Work?

By default, an LLM has no memory between calls — every message arrives as if it's the first. **Session state** solves this by maintaining a persistent context store tied to the conversation.

Conversation History

What it stores: The full thread of user and agent messages in order.

Lifetime: Active session only.

Key-Value State

What it stores: Named values explicitly set — e.g. user name, last product queried.

Lifetime: Configurable — in-memory or persistent.

How Values Get Into State

- A tool can write a value as a side effect of being called.
- The agent can be instructed to extract and store specific facts.
- State is read automatically on every subsequent turn.

In-Memory vs. Persistent

- **In-memory:** Lives only while the process runs — fast, simple, no infrastructure needed.
- **Persistent (e.g. Redis):** Survives restarts, scales across instances, enables resumable conversations.

 Session state is what turns a stateless model into a stateful assistant. Without it, every turn starts from zero.

9. How Do You Observe What an Agent Actually Did?

Agent behavior is not always transparent — the LLM's internal reasoning is hidden. A **trace** surfaces the full decision chain, step by step.

Example Trace

User message: "Where is my order #4521?"

- Intent identified: order status query — tool available
- Tool called: `get_order_status(order_id="4521")`
- Tool returned: `{ "status": "shipped", "eta": "2026-06-03" }`
- Reply generated: "Your order has been shipped and should arrive by June 3rd."

Three Failure Patterns a Trace Exposes Immediately

What the Trace Shows	What It Tells You
Tool called with wrong parameters	The instruction or tool description needs tightening
No tool called when one should have been	The tool is not registered, or its description is unclear
Correct tool result, wrong reply	The prompt synthesis logic needs refinement

📌 Always read the trace, not just the reply. The reply tells you what the agent said. The trace tells you **why**.

10. What Separates a Robust Agent From a Fragile One?

An agent that behaves well in testing often breaks in production. The difference is not capability — it's how the agent handles **everything outside the happy path**.

1. Graceful Unknowns

The agent says "I don't have that information" rather than guessing. Unanswerable queries return a useful fallback — not fabricated data.

2. Tool Failure Resilience

If a tool times out, returns an error, or sends malformed data, the agent still replies sensibly. Raw error messages never reach the user.

3. Scope Enforcement

The agent stays on topic even when users drift — or deliberately try to push it off course. Out-of-scope queries are declined politely, not silently ignored.

4. Consistent Persona

The agent sounds the same across all intents, all turns, and all edge cases. Tone and character do not break under pressure or unexpected input.

⚠ Robustness is not a final polish step. It is designed into the instruction and tool contracts **from the very beginning**.

What to Explore Next

If the concepts here felt solid, these questions point toward what comes next. Each one maps to a concrete technique you will work with.



Live Data Access

How do you give an agent access to live, real-time data — and what happens when that data source is unavailable?



Long-Term Memory

What does it mean for an agent to remember something permanently, not just within a single conversation?



Hallucination Prevention

How do you prevent an agent from confidently answering a question it has no reliable basis to answer?



Multi-Agent Architecture

When does a single agent become too complex to manage — and how do you think about splitting it?



Measuring Quality

How do you measure whether your agent is actually doing well, rather than just producing plausible-sounding replies?



Multi-Agent Tracing

When multiple agents work together, how do you trace which one made a decision — and why?